SISTEMAS DISTRIBUIDOS

Proyecto final: chat distribuido entre grupos

Escuela de Ingeniería de Sistemas e Informática · Universidad Industrial de Santander

Enunciado para estudiantes

1. Contexto y objetivos

Cada grupo construirá un servidor de chat en contenedores que se conecta con el servidor central del curso. A través de él, los usuarios de todos los grupos conversan en una misma sala global, de modo que el curso completo funcione como un único sistema de chat distribuido. El proyecto se trabaja en grupos de máximo 4 personas y se presenta con un póster científico y una sustentación de máximo 10 minutos.

Al terminar el proyecto, el grupo deberá ser capaz de:

- Implementar un servicio que cumple un protocolo común escrito por otra persona e interopera con implementaciones que no controla.
- Aplicar en un sistema real los conceptos de la Parte I del curso: enlaces fiables, descubrimiento de servicios y APIs.
- Manejar fallos parciales con reintentos, idempotencia, deduplicación y detección por heartbeat.
- Ordenar eventos sin reloj global mediante relojes de Lamport.
- Medir el comportamiento del sistema ante fallos con un experimento sencillo.
- Empaquetar y desplegar el sistema con contenedores de forma reproducible.
- Comunicar resultados técnicos en formato de póster científico.

2. Descripción general del sistema

El sistema tiene forma de estrella: un servidor central que opera el profesor y, a su alrededor, un servidor por cada grupo. Los servidores de grupo no se hablan directamente entre sí; todo mensaje pasa por el servidor central.

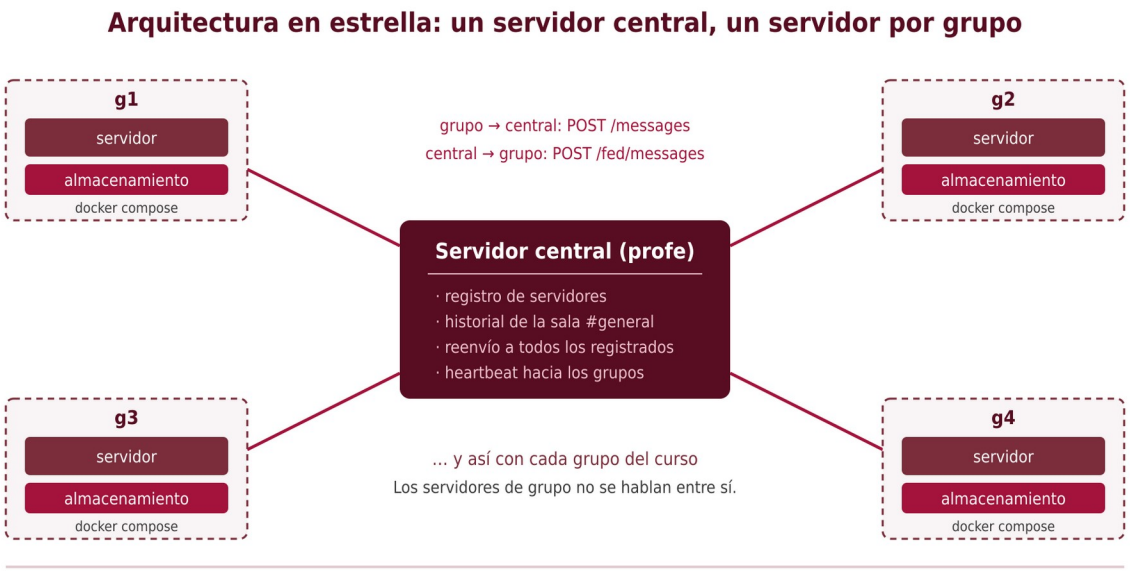


Figura 1. Arquitectura del sistema: servidor central y servidores de grupo.

- **Servidor central.** Lo opera el profesor. Lleva el registro de los servidores conectados, guarda el historial de la sala global `#general` y reenvía cada mensaje a todos los servidores registrados.
- **Servidor de grupo.** Cada grupo opera un servidor identificado por un nombre corto (`g1`, `g2`, …), con sus propios usuarios, que se escriben `usuario@gN`, por ejemplo `ana@g3`.
- **Sala global.** Hay una sola sala, `#general`, compartida por todo el curso. No hay salas propias de cada grupo.
- **Clientes.** Cada grupo entrega un cliente sencillo (línea de comandos o página web mínima) que habla únicamente con su propio servidor. La API entre el cliente y el servidor es libre; la API con el servidor central no.

Ejemplo de recorrido: `ana@g3` escribe en `#general`. Su servidor `g3` envía el mensaje al servidor central, que lo guarda en el historial y lo reenvía a `g1`, `g2` y `g4`. Cada servidor lo entrega a sus clientes.

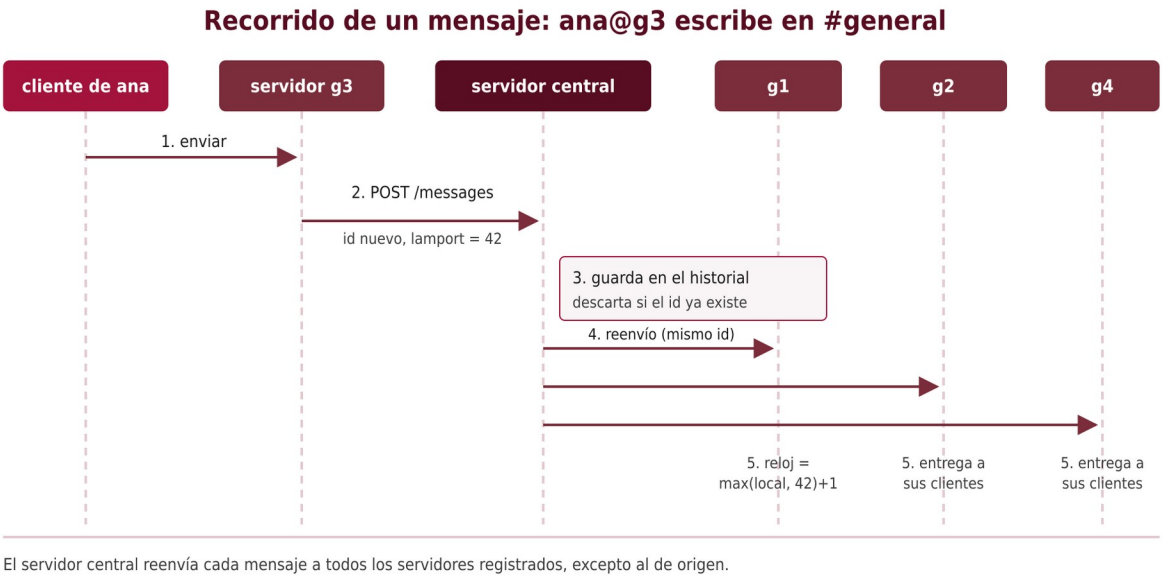


Figura 2. Recorrido de un mensaje a través del servidor central.

3. Requisitos del mínimo común

Todos los grupos deben cumplir estos requisitos. Son la condición para participar en la sesión de interoperabilidad.

1. **Despliegue en contenedores.** El sistema completo (servidor, almacenamiento y cualquier servicio auxiliar) se levanta con `docker compose up` en un equipo limpio. La configuración (nombre del grupo, URL pública, URL del servidor central) se pasa por variables de entorno.
2. **Persistencia.** Usuarios e historial sobreviven a `docker compose restart`. Puede usarse SQLite en un volumen, Redis con persistencia o PostgreSQL.
3. **Usuarios locales.** Crear usuarios del grupo y permitirles enviar y leer mensajes de `#general`.
4. **Registro.** Al arrancar, el servidor se registra en el servidor central y renueva su registro periódicamente. La URL del servidor central se lee de la configuración, nunca está escrita en el código.
5. **Envío y recepción.** Enviar al servidor central los mensajes de sus usuarios y recibir los que el servidor central reenvía, según la sección 4.
6. **Entrega al menos una vez.** Un mensaje que no pudo entregarse se reintenta con espera exponencial (backoff). El servidor descarta los duplicados por `id`, de modo que reintentar nunca produce mensajes repetidos en pantalla.
7. **Orden lógico.** Cada mensaje lleva una marca de reloj de Lamport. El servidor mantiene su reloj según las reglas vistas en clase y el historial se muestra ordenado por `(lamport, origin)`.
8. **Detección de fallos.** El servidor consulta `GET /health` del servidor central. Si no responde tras un número configurable de intentos, lo marca como caído, guarda los mensajes pendientes y los envía cuando vuelve.

9. **Recuperación de historial.** Al arrancar, o al volver de una caída, el servidor pide al servidor central lo que se perdió con `GET /history?since={lamport}`.
10. **Cliente.** Un cliente sencillo para enviar y leer mensajes, que distinga visualmente a los usuarios de otros grupos.
11. **Registro de eventos.** Logs legibles de lo que envía, recibe, reintenta y descarta el servidor. Son la base del experimento y de la demo.

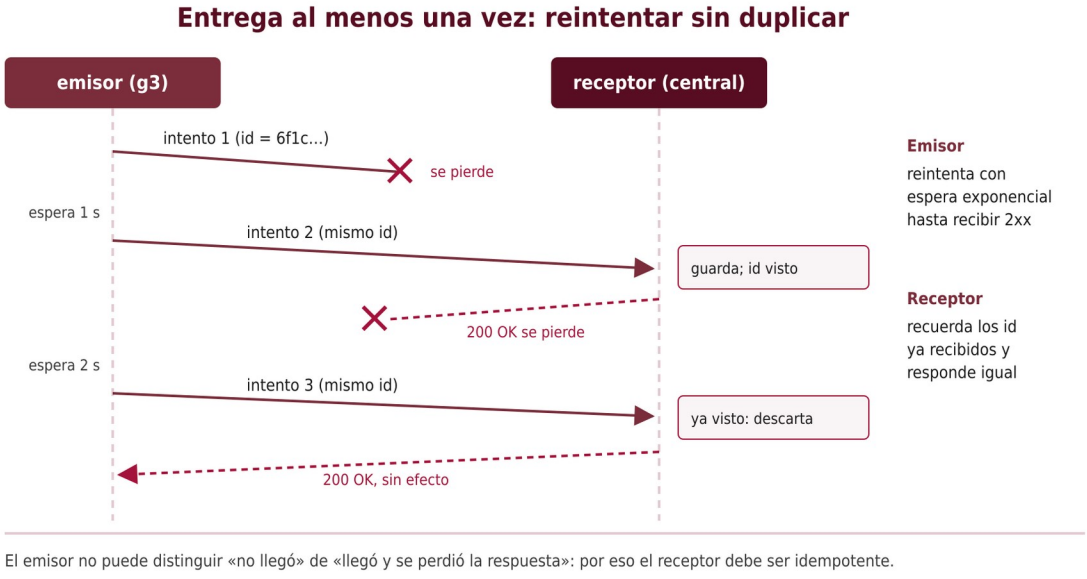


Figura 3. Reintentos del emisor y deduplicación en el receptor.

4. Protocolo (resumen)

La especificación completa se publica aparte en formato OpenAPI, junto con ejemplos JSON y el script de conformidad. Esta sección resume lo esencial; ante cualquier diferencia, manda la especificación. Todo el tráfico es HTTP con cuerpos JSON.

Endpoints del servidor central (los llama cada servidor de grupo)

Método y ruta	Propósito
`POST /register`	Registrar o renovar `{"server": "g3", "url": "http://…"}`
`POST /messages`	Entregar un mensaje de un usuario del grupo para la sala global
`GET /history?since={lamport}`	Pedir los mensajes posteriores a una marca de Lamport
`GET /health`	Heartbeat del servidor central

Endpoints de cada servidor de grupo (los llama el servidor central)

Método y ruta	Propósito
`GET /fed/health`	Heartbeat: responde con el nombre del servidor y su reloj de Lamport actual
`POST /fed/messages`	Recibir un mensaje reenviado por el servidor central

Formato de mensaje

```
{
  "id": "6f1c2a9e-4b7d-4f0e-9a51-2c8e7d3b1f44",
  "from": "ana@g3",
  "origin": "g3",
  "lamport": 42,
  "sent_at": "2026-11-10T14:03:22Z",
  "body": "¿Ya subieron el compose?"
}
```

Reglas de comportamiento

- `id` es un UUID generado por el servidor de origen y nunca cambia, ni siquiera al reenviar. Es la clave de deduplicación.
- Recibir un mensaje con un `id` ya conocido responde `200` sin efectos: la operación es idempotente.
- El servidor central reenvía cada mensaje a todos los servidores registrados, excepto al de origen.
- Antes de enviar, el servidor de grupo incrementa su reloj. Al recibir, toma `max(local, recibido) + 1`.
- `sent_at` es informativo: el reloj físico de cada máquina es distinto y nunca se usa para ordenar.
- Respuestas `5xx` o sin respuesta se reintentan; respuestas `4xx` no se reintentan y se registran en el log.
- Los campos desconocidos se ignoran sin error.

5. Experimento

El póster gira alrededor de una medición sencilla del sistema. Cada grupo elige una de estas preguntas:

Pregunta	Cómo se mide
Pérdidas ante una caída. ¿Cuántos mensajes se pierden o se duplican cuando el servidor del grupo se cae durante unos segundos?	Un script envía mensajes desde otro servidor a una tasa fija mientras el del grupo se apaga y se vuelve a levantar; después se cuentan los mensajes que faltan o sobran.

Pregunta	Cómo se mide
Latencia de extremo a extremo. ¿Cuánto tarda un mensaje desde que se envía hasta que aparece en otro servidor?	Dos servidores del grupo (`g3a` y `g3b`) en la misma máquina, para que compartan reloj; se mide la diferencia entre envío y recepción.
Tiempo de recuperación. ¿Cuánto tarda el servidor en ponerse al día después de reiniciarse?	Se reinicia el servidor y se mide el tiempo hasta que su historial coincide con el del servidor central.

El experimento debe incluir:

- **La pregunta elegida** y lo que el grupo espera observar.
- **Al menos 3 repeticiones** de la medición, con el resultado de cada una.
- **Una gráfica o una tabla** con los resultados, con unidades.
- **Los datos y el script de medición** en el repositorio.

Las caídas y reinicios pueden provocarse con `docker kill`, `docker compose restart` o el proxy `tcpspy.py` usado en el taller de sockets. Variar un parámetro (la duración de la caída, la tasa de envío) es opcional y suma en la evaluación.

6. Etapas de trabajo

El proyecto está pensado para una semana de trabajo. Las etapas no tienen tiempos asignados; cada una termina con un punto de control verificable, y conviene no avanzar a la siguiente sin haberlo alcanzado.

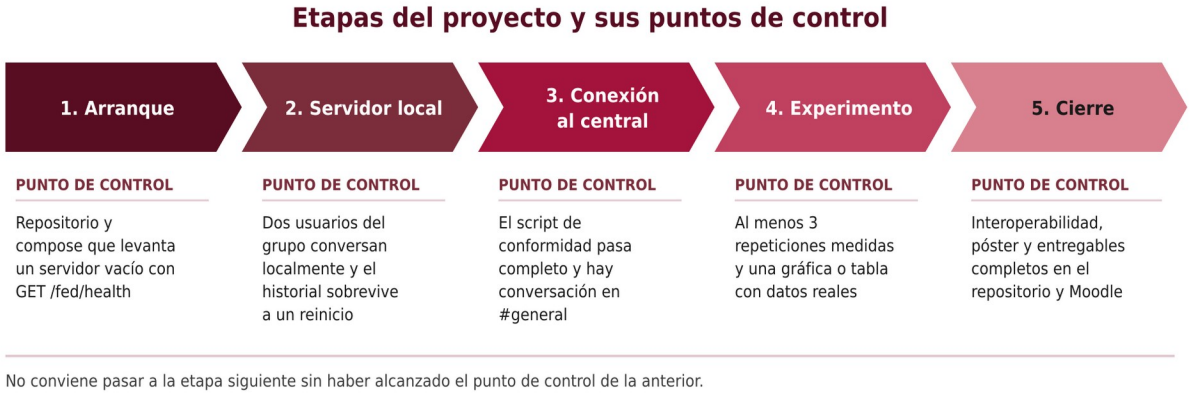


Figura 4. Etapas del proyecto y puntos de control.

1. **Arranque.**
   - Leer la especificación completa y resolver dudas en el foro del curso antes de escribir código.
   - Repartir roles dentro del grupo (servidor, conexión con el servidor central, experimento y póster) sin que nadie quede por fuera del código.
2. **Servidor local.**

  - Usuarios, mensajes y persistencia dentro del propio servidor.
  - Cliente sencillo funcionando.
3. **Conexión con el servidor central.**
  - Registro, envío y recepción, reloj de Lamport, deduplicación, reintentos, heartbeat y recuperación de historial.
  - Pruebas con el script de conformidad.
4. **Experimento.**
  - Elegir la pregunta, preparar el script de medición y medir.
5. **Interoperabilidad, póster y sustentación.**
  - Participar en la sesión de interoperabilidad (sección 7).
  - Diseñar el póster, ensayar la sustentación y la demo.

7. Sesión de interoperabilidad

En una sesión de clase todos los servidores del curso se conectan a la misma red y se registran en el servidor central. Es la prueba de que el sistema funciona como un todo y no solo dentro de cada grupo.

Durante la sesión se realizan, como mínimo, estas pruebas:

- **Conversación global.** Al menos un usuario de cada grupo escribe en `#general` y todos ven los mensajes de todos, en el mismo orden.
- **Caída y recuperación.** Por turnos, un grupo apaga su servidor mientras los demás siguen escribiendo; al volver, debe recuperar lo que se perdió sin mensajes duplicados.

Cada grupo lleva una bitácora de la sesión: qué funcionó, qué falló y por qué. Una incompatibilidad encontrada y bien explicada vale tanto como una prueba que pasó sin problemas; puede ir al póster como hallazgo.

Si la red de la sala no permite que el servidor central se conecte a los servidores de los grupos, se usará el modo alternativo que define la especificación. En ese caso el profesor avisa antes de la sesión.

8. Entregables

Repositorio

- Código fuente del servidor y del cliente, con `docker-compose.yml` funcional.
- `README.md` con instrucciones para levantar el sistema en un equipo limpio, variables de entorno y decisiones de diseño.
- Diagrama de arquitectura: contenedores, redes y volúmenes, y cómo se conectan con el servidor central.
- Script de medición y datos del experimento.
- Salida del script de conformidad y bitácora de la sesión de interoperabilidad.

- Historial de commits que muestre la participación de todos los integrantes.

Póster científico

Formato vertical, tamaño pliego o A1, entregado también en PDF. Debe contener:

1. Título, integrantes y grupo (`gN`).
2. Pregunta e hipótesis.
3. Arquitectura: diagrama de contenedores y de la conexión con el servidor central.
4. Método: cómo se hizo la medición y cuántas repeticiones.
5. Resultados: una gráfica o tabla con unidades.
6. Interoperabilidad: qué funcionó en la sesión y qué se encontró.
7. Limitaciones y conclusiones.
8. Referencias, incluido el libro de texto.



Figura 5. Distribución sugerida de las secciones del póster.

El póster se lee de pie, en pocos minutos: más figuras que texto, frases cortas y ningún bloque de código largo.

Sustentación

Máximo 10 minutos frente al póster:

- Presentación del póster: pregunta, diseño y resultados.
- Demo en vivo, idealmente provocando un fallo y mostrando la recuperación.
- Preguntas del profesor, que pueden dirigirse a cualquier integrante sobre cualquier parte del sistema.

9. Evaluación

La nota del proyecto se compone así. El mínimo común se verifica en buena parte con el script de conformidad, de modo que el grupo conoce de antemano el resultado de esa parte.

Componente	Peso (%)	Qué se espera para la nota máxima
Mínimo común	40	El script de conformidad pasa completo; el sistema se levanta con un solo comando en un equipo limpio; la persistencia, los reintentos, la deduplicación y el heartbeat funcionan ante fallos reales
Experimento	15	La pregunta está bien planteada, hay al menos 3 repeticiones, los resultados tienen unidades y el grupo explica lo que obtuvo
Interoperabilidad	15	El servidor participa en las pruebas de la sesión y la bitácora explica las fallas encontradas, propias o ajenas
Póster	15	Contiene todas las secciones, se entiende sin el grupo presente y las figuras dominan sobre el texto
Sustentación y demo	15	Respeta los 10 minutos, la demo muestra un fallo y su recuperación, y cualquier integrante responde sobre cualquier parte
Total	**100**	

La nota es grupal, pero el profesor puede ajustarla individualmente según las respuestas en la sustentación y el historial de commits.

10. Reglas y recomendaciones

Reglas

- El protocolo no se negocia entre grupos. Las dudas se resuelven en el foro del curso, donde la respuesta del profesor vale para todos.
- Está permitido, y es deseable, avisar a otros grupos de los errores que se observen en la sala global. No está permitido compartir código entre grupos.
- Puede usarse cualquier lenguaje o framework para el servidor, siempre que corra en contenedores y cumpla la especificación. Se recomienda Python con FastAPI, que ya conocen del curso.
- El uso de asistentes de IA está permitido. Cada integrante debe poder explicar cualquier línea del código entregado.

Recomendaciones

- Empiecen por la versión más simple que pase el script de conformidad. Mejorar después algo que funciona es mucho más fácil que depurar a última hora algo ambicioso.

- Pongan el log en el centro: cada mensaje enviado, recibido, reintentado o descartado debe dejar rastro con su `id` y su marca de Lamport. Es su mejor herramienta de depuración y la fuente de datos del experimento.
- Levanten dos servidores en su propia máquina con distintos nombres (`g3a`, `g3b`) para probar sin depender de otros grupos.
- Provoquen los fallos con scripts, no a mano: así la medición se puede repetir.
- Diseñen el póster a medida que obtienen resultados, no al final.
- Si trabajan en la sala de cómputo, verifiquen con anticipación que Docker esté disponible y que los equipos se vean en la red; si no, trabajen en sus portátiles.